

DuckDB S3 MinIO

Integración DuckDB S3 MinIO desde Python

Amazon S3 es una solución de almacenamiento en la nube que es altamente utilizada en infraestructuras como el [Data Lakehouse](#). La [integración de S3 y DuckDB](#) es paradigma de eficacia y simplicidad

```
import duckdb

%load_ext sql
%config SqlMagic.autopandas = True
%config SqlMagic.displaycon = False
conn = duckdb.connect()
%sql conn --alias duckdb

duckdb.execute("INSTALL httpfs")
duckdb.execute("LOAD httpfs")
```

Extensiones: DuckDB Secrets Manager

Las [credenciales](#) son tratadas como tipos de datos en *DuckDB*. En su mayoría son registradas como **extensiones** que pueden ser de dos tipos: [core](#) (gestionadas por DuckDB) y [community](#) (mantenidas por terceros)

En la presente documentación únicamente se va a tratar con las primeras siendo especialmente relevantes (en orden alfabético) las cuatro siguientes:

- [avro](#): lectura de datos en formato [Apache Avro](#)
- [https](#): acceso a sistemas de ficheros remotos a través de las *APIs* [HTTP\(S\)](#) y [S3](#)
- [MySQL](#): integración con *MySQL* y *MariaDB*
- [parquet](#): acceso a ficheros en formato [Apache Parquet](#) contemplando [particionado Hive](#)

```
%%sql
CREATE OR REPLACE SECRET secret (
  TYPE s3,
  PROVIDER config,
  KEY_ID 'lo_que_sea',
  SECRET 'lo_que_sea',
  -- URL Izar MinIO Server
  ENDPOINT 'izar.ls.fi.upm.es:30009',
  URL_STYLE 'path',
  REGION 'eu-southt-2',
  USE_SSL true
  -- USE_SSL false
);
```

```
%%sql
-- Mostrar secrets
FROM duckdb_secrets()
```

	name	type	provider	persistent	storage	scope	secret_string
0	secret	s3	config	False	memory	[s3://, s3n://, s3a://]	name=secret;type=s3;provider=config;serializab...

```
%%sql
-- OJO: la BD está almacenada en el servidor de objetos
ATTACH 's3://sakstar/WindowFunction.duckdb' AS wfdb;
```

```
%%sql
SHOW ALL TABLES;
```

	database	schema	name	column_names	column_types	temporary
0	wfdb	main	products	[product_id, product_name, model_year, list_pr...	[BIGINT, VARCHAR, BIGINT, DOUBLE]	False

```
%%sql
SELECT model_year, product_name, list_price,
  AVG(list_price) OVER (PARTITION BY model_year) avg_price
```

```
FROM wfdb.products
ORDER BY product_name;
```

	model_year	product_name	list_price	avg_price
0	2018	Electra Amsterdam Fashion 3i Ladies' - 2017/2018	899.99	1658.470441
1	2017	Electra Amsterdam Fashion 7i Ladies' - 2017	1099.99	1279.931176
2	2017	Electra Amsterdam Original 3i - 2015/2017	659.99	1279.931176
3	2017	Electra Amsterdam Original 3i Ladies' - 2017	659.99	1279.931176
4	2018	Electra Amsterdam Royal 8i - 2017/2018	1259.90	1658.470441
...	...	...	...	...
316	2017	Trek X-Caliber 8 - 2017	999.99	1279.931176
317	2018	Trek X-Caliber 8 - 2018	999.99	1658.470441
318	2018	Trek X-Caliber Frameset - 2018	1499.99	1658.470441
319	2018	Trek XM700+ - 2018	3499.99	1658.470441
320	2018	Trek XM700+ Lowstep - 2018	3499.99	1658.470441

Por último, el ejemplo siguiente muestra una consulta sobre un archivo en formato CSV almacenado en el servidor de objetos

```
%%sql
SELECT *
FROM read_csv_auto('s3://sakstar/dim_date.csv')
LIMIT 10;
```

	date_key	date_actual	epoch	day_suffix	day_name	day_of_week	day_of_month	day_of_quarter	day_of_year	week_of_month	...	last_day_of_w
0	19700101	1970-01-01	0.0	1st	Thursday	4	1	1	1	1	...	1970-01-04
1	19700102	1970-01-02	86400.0	2nd	Friday	5	2	2	2	1	...	1970-01-04
2	19700103	1970-01-03	172800.0	3rd	Saturday	6	3	3	3	1	...	1970-01-04
3	19700104	1970-01-04	259200.0	4th	Sunday	7	4	4	4	1	...	1970-01-04
4	19700105	1970-01-05	345600.0	5th	Monday	1	5	5	5	1	...	1970-01-11
5	19700106	1970-01-06	432000.0	6th	Tuesday	2	6	6	6	1	...	1970-01-11
6	19700107	1970-01-07	518400.0	7th	Wednesday	3	7	7	7	1	...	1970-01-11
7	19700108	1970-01-08	604800.0	8th	Thursday	4	8	8	8	2	...	1970-01-11
8	19700109	1970-01-09	691200.0	9th	Friday	5	9	9	9	2	...	1970-01-11
9	19700110	1970-01-10	777600.0	10th	Saturday	6	10	10	10	2	...	1970-01-11

```
%%sql
DETACH wfdb;
```

```
# Siempre, siempre, siempre cerrar las conexiones
duckdb.close()
```